Code-Layout, Readability and Reusability

Date	29 September 2026
Team ID	SWTID-2026-8642
Project Name	Import Data Using Transform Maps (Spreadsheet)
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard PEP 8 (4 spaces) used across Python scripts and JS files.
2	Proper File Structure	Files and folders are logically organized	Yes	Modular directory setup (/controllers, /services, /models, /transform_scripts).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables named intuitively (source_spreadsheet_df, target_schema_fields, coalesce_key_matched).
4	Function / Method Names	Functions are descriptively named	Yes	Functions named using action verbs

				(parse_excel_file(), apply_transform_map(), validate_coalesce_rules()).
5	Code Comments	Inline and block comments explain log	Yes	Docstrings and inline comments added to explain complex mapping logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	File ingestion, transform mapping engine, and database writer modules are decoupled.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Formatted using Flake8 and ESLint to ensure no unused imports or redundant code.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks implemented for missing file headers, corrupt CSV rows, and database write errors.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Spreadsheet Parser Engine	Python (pandas / openpyxl)	Data Ingestion Service & Data Validation Module	High
2	Field Mapping Engine	JavaScript / Python	Dynamic UI Field Mapper & Background Batch Import Process	High

3	Coalesce Matcher & Deduplicator	Python (pandas)	Pre-Insert Database Validator & Audit Service	High
4	Execution Log Generator	Python / SQL	Error Logging Dashboard & Admin Email Notification Alert	Medium
5	File Upload Dropzone Component	React.js / HTML5	User File Upload Screen & Admin Template Re-upload Modal	High

Overall Code Quality Assessment:

Aspect	**Rating (1-5)**	**Comments**
Code Layout & Structure	5	Well-organized folder structure separating business logic, API routes, and file parsing tasks.
Readability	4.8	Highly readable code following standard coding guidelines and descriptive variable names.
Reusability	5	Core components like file parsers and mapping engines are fully generic and reusable in other import modules.
Documentation / Comments	4.5	Clean function docstrings and inline comments provided for all complex data transform scripts.
Overall Score	4.8/5	Production-ready codebase with clean structure, high reusability, and robust error handling.